

Neural Operator-Based Surrogate Solver for Free-Form Electromagnetic Inverse Design

Yannick Augenstein,* Taavi Repän, and Carsten Rockstuhl*



Cite This: *ACS Photonics* 2023, 10, 1547–1557



Read Online

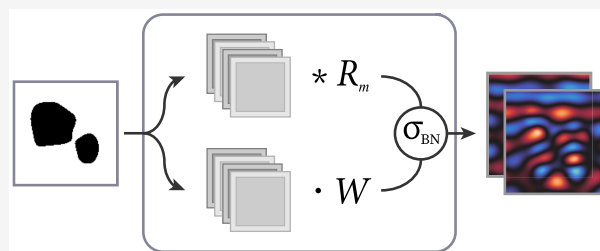
ACCESS |

Metrics & More

Article Recommendations

ABSTRACT: Neural operators have emerged as a powerful tool for solving partial differential equations in the context of scientific machine learning. Here, we implement and train a modified Fourier neural operator as a surrogate solver for electromagnetic scattering problems and compare its data efficiency to existing methods. We further demonstrate its application to the gradient-based nanophotonic inverse design of free-form, fully three-dimensional electromagnetic scatterers, an area that has so far eluded the application of deep-learning techniques.

KEYWORDS: machine learning, neural operators, inverse design, nanophotonics



INTRODUCTION

Tools for solving Maxwell's equations are essential in nanophotonics for modeling light-matter interaction at the wavelength scale and, in extension, for designing new optical devices. Some of the most established methods for this purpose are finite element solvers (FEM)^{1,2} in the frequency domain and finite difference solvers for both frequency (FDFD)^{3,4} and time domain (FDTD).^{5–7} As full-wave Maxwell solvers, these methods represent the most general and accurate class of tools for modeling electromagnetic systems. However, full-wave solutions often involve significant time and computational cost, placing a practical limit on the scale of problems that can be tackled. This is exacerbated in problems such as inverse design,^{8–12} where typically on the order of hundreds of simulations have to be performed to reach reasonable solutions. While there are ongoing developments in the search for faster full-wave solvers,^{13,14} there exist also a variety of semianalytical methods^{15–18} that can potentially offer order-of-magnitude speedups. However, such methods make specific physical assumptions, and their applicability is, therefore, generally limited to certain problem classes.

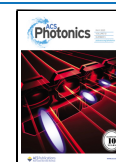
A more recent development has been the use of machine-learning-based surrogate models^{19–21} to approximate solutions to partial differential equations (PDEs). Such models can, during inference, be as fast or faster than semianalytical methods and can, in principle, be tailored to suit a wide variety of problems because of their property as universal approximators.^{20,22} Machine learning has garnered significant momentum in nanophotonics in recent years,^{23,24} and surrogate solvers have been applied to both forward modeling and inverse design.^{25–29} Of course, such models bring their

own challenges and limitations, among which the main ones are loss of accuracy and data efficiency. As data efficiency implies using less data to achieve more, these two quantities have an inverse relationship; at the cost of using more data, a model can generally be trained to produce higher accuracy predictions and vice versa. As high-quality training data is usually generated using classical methods, e.g., full-wave solvers, it is crucial to consider this trade-off in any application that uses surrogate solvers since they quickly lose their edge over conventional methods when the cost of generating training data limits them. This is exacerbated by the fact that surrogate models can not serve as a general-purpose tool for solving scattering problems; instead, they are limited in scope to the type of problem modeled in the training data. It is, therefore, essential to increase the data efficiency of such models and find applications where the use of surrogate solvers can maximally offset the cost of data generation.

The key to improving the data efficiency of surrogate solvers lies in the development and application of new models in the context of scientific machine learning. One aspect that has proven effective is imparting some partial physical knowledge to the surrogate model.³⁰ This can either be done by explicitly including governing equations into the model formulation, such as in the case of physics-informed neural networks (PINNs)^{31,32} or implicitly via penalizing unphysical solutions

Received: February 2, 2023

Published: March 29, 2023



during training.^{29,33} The latter is not tied to specific models and can generally be applied when the governing equations (or some of their properties) are known. Parallel to the incorporation of physics information into machine-learning models has been the development of new classes of model architectures. In particular, learning operator mappings between function spaces using deep neural networks has been repeatedly shown to outperform previous approaches on PDE-constrained problem sets.^{34–37} Such models include graph kernel networks (GKN),³⁴ deep operator networks (DeepONets),^{20,38} and Fourier neural operators (FNO),³⁵ among others.

However, even the most data-efficient models can not offset the cost of data generation if the problem at hand requires only a handful of simulations in the first place. Therefore, we identify inverse design,^{39–42} specifically gradient-based, as a discipline that is well-suited to benefit from the speed of surrogate models and suffers little from their drawbacks.^{29,43} In gradient-based inverse design, a functional element is optimized by incrementally maximizing some figure of merit. Gradients of this figure of merit with respect to incremental changes in the geometry are then used to refine the device until an optimum is found iteratively. Conventional inverse design, e.g., topology optimization,⁴⁴ typically takes a few hundred iterations to converge, where each iteration entails two full-wave simulations—one for evaluating the figure of merit and one for obtaining its gradients via the adjoint method. Depending on the problem, a single optimization in 3D can easily take days to complete, and the solutions generally depend strongly on the initial conditions, i.e., ideally, many such optimizations should be performed to judge the quality of the optimization. Further, such optimizations generally can not be parallelized, as the incremental updates are serial in nature. As such, the only remaining possibility to accelerate the inverse design process is to speed up the individual simulations.

Neural network-based surrogate solvers seem ideally suited to tackle this challenge, their inference time is negligible when compared to full-wave simulations, training samples are independent and can be generated in parallel, and they are inherently *differentiable*, meaning they can generally be used as a drop-in replacement for differentiable Maxwell solvers^{4,45} in existing gradient-based optimization pipelines. At the same time, their drawbacks are mitigated to an extent. During an optimization, the simulation parameters are generally fixed, and only the device geometry varies, meaning that it is feasible to use specialized models trained on as little data as possible and stay within the training distribution during optimization. Furthermore, for many inverse design problems, high numerical accuracy is often not crucial to reach a feasible solution, especially considering fabrication uncertainty that is typically on the order of a few percent. In cases where high accuracy is required, an initial device design (obtained using low numerical accuracy) can often be refined using high-accuracy methods within a few additional iterations.

In this work, we implement a modified version of FNO as a surrogate solver for electromagnetic scattering problems. The model is trained on a diverse set of free-form electromagnetic scatterers. We compare its performance to a state-of-the-art convolutional architecture (UNet) in a simplified two-dimensional setting. We show that FNO requires significantly less data to reach the same accuracy as UNet. In the inverse design section, we use this model for the gradient-based inverse design of free-form scatterers in three dimensions from

many different initial conditions, a task that is extremely computationally demanding using full-wave solvers.

We provide full access to our code as well as training data sets and model weights in accordance with the FAIR data principles;⁴⁶ see the data availability section for further details.

■ FOURIER NEURAL OPERATOR FOR ELECTROMAGNETIC FIELD INFERENCE

We begin with a brief description of the vanilla Fourier neural operator (FNO) introduced in Li et al.³⁵ and outline some architectural choices particular to this work. The core idea behind the FNO architecture is learning a kernel parametrized in Fourier space, where each Fourier layer in the FNO performs a *global* convolution on its input. First, the input $v(x)$, where x denotes a location on the computational mesh, is lifted to a higher dimensional representation $y(x)$ with

$$y(x) = C_{\text{in}}(v(x)) \in \mathbb{R}^w \quad (1)$$

where $C_{\text{in}}: \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^w$ is a transformation parametrized by a linear layer with the input dimension d_{in} and the output dimension w . While the dimension of the input d is determined by the problem and is typically small, the output dimension w is a hyperparameter termed “width” of the FNO and corresponds to the number of kernels (feature channels) in each Fourier layer. After lifting the input dimension, the data is passed through a sequence of n “Fourier blocks”. Each such block consists of the Fourier layer $\kappa_m(y)$, a linear update, and a batch normalization layer,⁴⁷ followed by an activation function (ReLU) σ_{BN} :

$$u(y) = \sigma_{\text{BN}}(\underbrace{\mathcal{F}^{-1}(R_m \cdot \mathcal{F}(y))}_{\kappa_m(y)} + Wy + b) \quad (2)$$

with the Fourier transform $\mathcal{F}$, its inverse $\mathcal{F}^{-1}$, and the trainable parameters R_m , W , and b . Together, W and b form the linear layer update, where W is a weight matrix that acts locally on y , and b is the bias vector. The complex-valued tensor R_m represents the kernel matrix of the convolution in Fourier space. After the Fourier blocks, y is mapped to the desired output dimensionality d_{out} via a linear layer:

$$z(x) = C_{\text{out}}(y(x)) \in \mathbb{R}^{d_{\text{out}}} \quad (3)$$

A defining feature of FNOs is that the Fourier layer $\kappa_m(y)$ truncates the Fourier series at the m -th Fourier coefficient, i.e., R_m only contains entries up to the frequency index of that mode. This truncation has some important implications on the expressiveness of an FNO network. Most importantly, it acts as a low-pass filter by construction, and high-frequency spatial components are strongly suppressed in the output, leading to relatively smooth outputs (depending on the choice of m). It turns out that this is advantageous in the context of many PDEs that describe physical systems, in particular for systems that have wave-like solutions such as Maxwell’s equations. This makes FNOs uniquely suited to model these systems, as this property is an inherent part of the architecture and does not need to be enforced through other means, such as explicitly smoothing the output or modifying the loss function; both approaches are technically feasible, but introduce additional complexity. Moreover, approaches that modify the loss function, such as the ones used in Chen et al.²⁹ or Lim and Psaltis,³³ which aim to make the model predictions more self-consistent with Maxwell’s equations, can be readily incorporated in the training of FNOs if necessary. Here, we are only

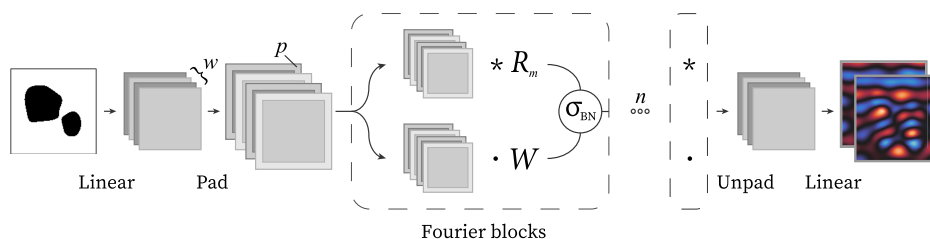


Figure 1. Illustration of the FNO architecture. The input is expanded via a linear layer to w channels and then zero-padded by p pixels. Data is then passed through a series of n “Fourier blocks”, each containing a linear layer and a convolution in Fourier space using the learned kernel R_m , a batch normalization layer as well as a GELU nonlinearity. Finally, the padding is removed, and the channel dimensions are reduced via a linear layer to the desired output dimensions.

concerned with establishing a baseline performance for FNOs in the context of electromagnetic scattering problems.

In practice, vanilla FNOs use feature expansion on the inputs to achieve good accuracy.^{35–37} The canonical choice is to add a Cartesian coordinate grid as an additional input feature, i.e., the coordinates in each spatial dimension are added to the input channels, and the input dimensionality becomes, e.g., $\mathbb{R}^d \rightarrow \mathbb{R}^{d+3}$ in three dimensions. We have found that similar accuracy can be achieved by zero-padding the input ($p = 2$) before passing it through the Fourier blocks and removing the padding again before the final layer of the network, eliminating the need for feature expansion. Further, we use the Gaussian Linear Error Unit (GELU) activation function⁴⁸ instead of ReLU, which we have empirically found to lead to slightly lower prediction errors. A pictorial representation of the FNO architecture used here is shown in Figure 1.

RESULTS

Our aim is to use FNOs as surrogate solvers for Maxwell’s equations in place of full-wave methods such as the finite-difference time-domain (FDTD) method for some particular scattering problem. To do this, we define the input of the FNO to be a dielectric material distribution that is discretized on a regular grid, meaning that the number of input features $d_{\text{in}} = 1$. The output should then contain the complex electromagnetic field components of interest, where each field component uses two output channels for the real and imaginary parts, respectively. The FNOs discussed in this work are fully described by the set of hyperparameters in Table 1.

Table 1. Hyperparameters Describing the FNO Architecture^a

parameter	description	value
n	No. of Fourier blocks	10
m	Fourier modes (truncation order)	12
w	hidden channels (width)	32
p	zero-padding in each spatial direction	2

^aThe column “value” contains the parameter values for the networks discussed in this work (both in 2D and 3D).

To train the models, we generate data sets of scatterers and fields using a full-wave Maxwell solver. A detailed description of the data generation is given in the data generation section. As a training and evaluation metric, we use the normalized L_p loss

$$L_p(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^n \frac{\|y_i - \hat{y}_i\|_p}{\|y_i\|_p} \quad (4)$$

throughout. The models are trained on the normalized L_2 loss, i.e., normalized root-mean-square error. However, we will generally refer to the normalized L_1 loss for evaluation and discussion due to its more intuitive interpretability as the absolute error between two samples. Further, we note that the discrepancy between L_1 and L_2 is minimal for the examples in this work, and in practice, the two can be used almost interchangeably. For evaluation, we use a separate test set containing 400 samples not included in the training data (neither for training nor validation). When referring to specific models, we use the name “FNO-2D” for models trained on two-dimensional data and “FNO-3D” for models trained on three-dimensional data. For remarks and observations that apply more generally, we will use the term “FNO” as before.

Field Inference. Previous works have focused on UNet-like⁴⁹ convolutional architectures for electromagnetic field inference.^{27,29,33} To make an objective comparison between UNet and FNO performance for this task, we use the UNet architecture from Chen et al.²⁹ as a reference implementation and train both models on data sets of simulations of a diverse set of random scatterers in two dimensions and vary the number of training samples to judge their respective data efficiency, i.e., the number of training samples needed to reach a certain prediction accuracy. Here, both networks use two output channels that contain the real and imaginary parts of the z -component E_z of the electric field, respectively. Both models are trained under the same conditions, and details of the training procedure are given in the FNO and UNet training section.

The evaluation results on the 400-sample test set are visualized in Figure 2, and numerical results are provided in Table 2.

While both network architectures see a significant reduction in test error for an increasing number of training samples, we observe that FNO-2D consistently outperforms UNet. Not only does FNO-2D have significantly better prediction accuracy, the distribution of errors on the test set shows a much tighter spread around the mean, indicating that FNO-2D not only has a higher prediction accuracy overall but also generalizes better to samples that are far from the data set mean. This is particularly remarkable considering that FNO-2D has only 5909250 trainable parameters in total, compared to UNet, which has 23617970 trainable parameters, meaning that FNO-2D is significantly more parameter-efficient. To reach a test error of around 5%, FNO-2D requires 4096

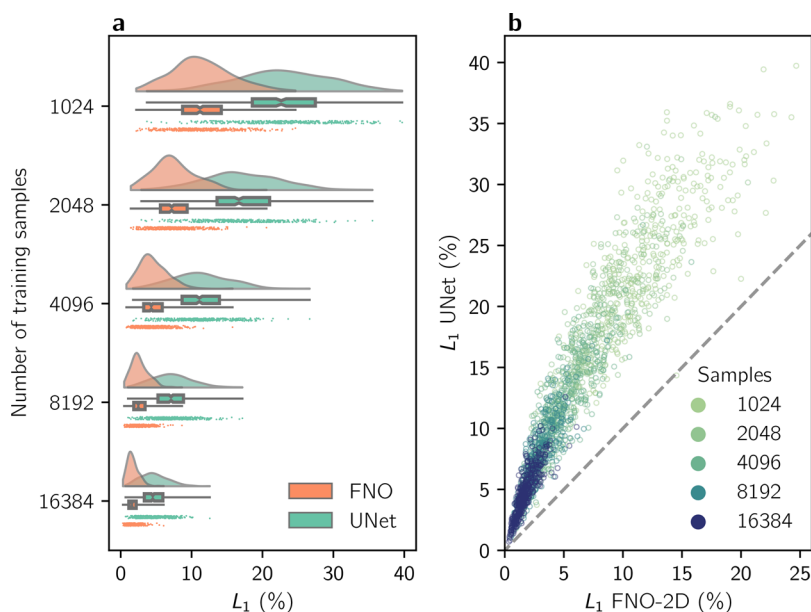


Figure 2. (a) Raincloud plot⁵⁰ of the normalized L_1 errors on a test set with 400 samples for FNO-2D and UNet trained on varying training set sizes. Each set of training samples shows a kernel density estimate (top, using Scott's rule for bandwidth estimation⁵¹), a box plot of the distribution with a notch indicating the median (middle), as well as a categorical scatter plot containing all samples (bottom). (b) Comparative scatter plot of the L_1 errors for all test samples and training set sizes of FNO-2D and UNet. The dashed gray line indicates where UNet and FNO-2D errors are equal.

Table 2. Median Normalized L_1 and L_2 Errors, as Measured over 400 Test Samples for Networks Trained on a Varying Number of Training Samples^a

samples	network	L_1 (%)	L_2 (%)	$\sigma(L_2)$ (%)	time per epoch ^b (s)
1024	FNO-2D	11.18	12.07	4.39	8.31
	UNet	22.64	24.42	6.80	10.15
2048	FNO-2D	7.19	7.72	3.15	10.10
	UNet	16.68	18.01	5.59	13.24
4096	FNO-2D	4.32	4.64	2.12	13.72
	UNet	11.09	11.92	4.15	18.96
8192	FNO-2D	2.48	2.65	1.30	21.13
	UNet	7.10	7.58	2.88	30.92
16384	FNO-2D	1.59	1.68	0.89	35.81
	UNet	4.52	4.84	1.99	55.26

^aThe lowest error achieved on each training set size is marked **bold**.

^bTiming was performed on a single NVIDIA A100 SXM4 GPU.

training samples, whereas UNet requires four times as many to reach a comparable value. As shown in Figure 2a, this observation holds over the whole range of training set sizes that we considered, where UNet generally lags behind FNO-2D performance by a factor of 4 with respect to training set size. We note that these comparative results are broadly in line with previous findings on benchmarking FNO and UNet performance on PDE-constrained problem sets.^{35,36} Furthermore, for the same number of samples, FNO-2D training is roughly 30% faster than UNet, with the gap widening slightly for a larger number of training samples. This is, however, mostly because FNO-2D has significantly fewer parameters than UNet—in fact, it is slower than UNet when training time is considered per parameter.

It is worth investigating whether the lower test error of FNO-2D holds for all samples or if, while having a lower error on average, some samples are nonetheless better predicted by UNet. Such a scenario might indicate that for specific

geometries, choosing UNet over FNO-2D could still be advantageous. A comparison between FNO-2D and UNet L_1 losses for each test sample and all training runs is visualized in the scatter plot in Figure 2b, where the dashed line marks the line where FNO-2D and UNet errors are equal. From this, we can clearly see that FNO-2D outperforms UNet for *all* test samples irrespective of training set size.

Figure 3 shows a typical sample drawn from the test set, evaluated on the FNO-2D and UNet models trained on 16k samples. The sample is chosen to be as close as possible to the median FNO-2D L_1 error on the test set and has a relative L_1 error of 1.58% for the FNO-2D prediction and an error of 4.29% for the UNet prediction. We can see a good qualitative agreement between the fields in Figure 3a–c for both models. This picture changes, however, when we consider the absolute error maps between the ground truth and the field predictions in Figure 3d,e. We observe that the UNet prediction has a significantly higher deviation throughout, which of course, is already clear from the data presented. Somewhat more interestingly, we note that the error map produced by the FNO-2D model in Figure 3e shows less spatial noise when compared to the one produced by the UNet model in Figure 3d, i.e., the map appears smoother overall. This is in line with what one would expect from these two architectures, while convolutional neural networks perform pixel-wise local convolutions, FNO instead performs *global* convolutions and explicitly discards Fourier components above a certain order by design. Essentially, the priors built into the FNO architecture force the model to learn continuous solutions, a property that is later reflected in its output.

Inverse Design. We will now demonstrate the inverse design of free-form, three-dimensional nanophotonic devices using an FNO model trained on a data set of 8192 pairs of volumetric scatterers and fields. The data generation and FNO-3D training are detailed in the data generation section and the FNO and UNet training sections. After 100 epochs of training,

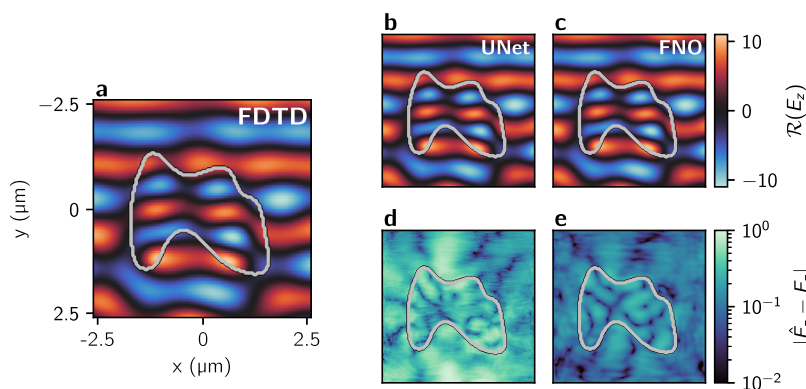


Figure 3. Real part of the (a) ground truth, (b) UNet predicted, and (c) FNO-2D predicted electric field E_z for a random scatterer drawn from the test set. Also shown are the absolute errors between the predicted fields and the ground truth for the (d) UNet prediction and (e) FNO-2D prediction. FNO-2D has a normalized L_1 error of 1.58%, and the UNet model has a normalized L_1 of 4.29% on the sample. Both networks were trained on the same data set containing 16k training samples. The outline of the scatterer is shown in all plots. The samples are illuminated with a plane-wave source impinging from the negative y -direction.

the normalized L_1 loss of the FNO-3D model on the 400-sample test set is 5.05%.

The inverse design process is relatively straightforward, as we use FNO-3D in place of a Maxwell solver; the objective function can still be an arbitrary (differentiable) function of the fields. However, our formulation differs from the typical density-based approach to inverse design in how the scatterers are parametrized. Instead of optimizing the scatterers directly on the computational grid, they are represented in the latent space of a pretrained variational autoencoder (VAE).⁵² The reason for this is the gradient-based nature of the design process; parametrizing the scatterers directly on the grid would lead to continuous variations in the permittivity of the scatterer. However, we train FNO-3D only on binary data, meaning that field inference will fail for scatterers with permittivities outside of this range. While it is straightforward to include more permittivity values in the training data, this would require a fine sampling between the lowest and highest permissible permittivity for continuous inverse design, which would lead to a significant increase in data set size, defeating the purpose of using a specialized surrogate solver for fast inverse design. Instead, we train a convolutional VAE that generates random scatterers from the same distribution from which the training set was generated. Afterward, the device of interest can be optimized directly in the latent space of the VAE. Crucially, the latent space vector can be continuously updated, and the VAE decoder will map this to (approximately) binary geometries for which FNO-3D can give accurate predictions. The encoder part of the VAE is only needed during training and is not used during inverse design. Note that data generation for VAE training is very cheap as there is no simulation involved; in fact, the data is generated on-the-fly during training. Details of the VAE architecture and training are given in the VAE setup and training section. Note that the VAE parametrization is not a requirement for using FNO for inverse design; in principle, any parametrization that yields binary scatterers can be used, e.g., an explicit geometrical or boundary parametrization. A pictorial representation of the inverse design pipeline is shown in Figure 4.

As a demonstration, we optimize two nanophotonic devices using FNO-3D. We define a simple objective function in terms of the electric field:

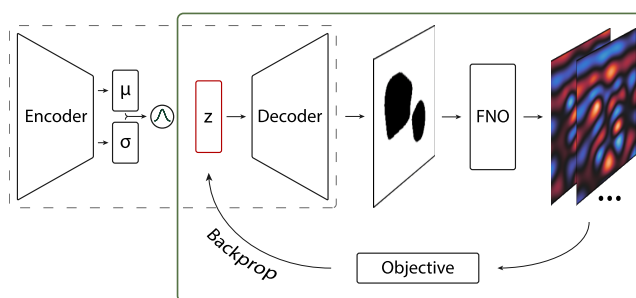


Figure 4. Flowchart of the iterative inverse design procedure. The decoder of a pretrained variational autoencoder (dashed gray box) is used to generate scatterers from the latent vector z (red box). The electric field is then obtained via the FNO model, from which the loss is calculated. Gradients of the objective with respect to the latent variables are obtained via backpropagation, from which they are then iteratively updated until the optimization converges.

$$J(\mathbf{E}) = \sum_{\mathcal{D}} |\mathbf{E}(\mathbf{r})|^2 \quad \forall \mathbf{r} \in \mathcal{D} \quad (5)$$

where the domain $\mathcal{D}$ represents the spatial points at which we wish to maximize the electric field intensity. For the first device, we maximize the intensity at a single focal spot centered in the x - y plane, i.e., we design a simple nanophotonic lens. In the second example, we maximize the intensity at four focal spots, each centered in one quadrant in the x - y plane. The plane wave illumination spans the x - y plane and impinges from the top ($z = 0 \mu\text{m}$) and the focal plane lies at the opposite end at $z = 4.8 \mu\text{m}$. We optimize both devices for 300 iterations using AdamW.⁵³ In principle, other optimization algorithms such as L-BFGS-B⁵⁴ or MMA⁵⁵ can be used; however, we choose AdamW here because of readily available GPU implementations and support for optimizing multiple devices in parallel. The results for these optimizations are shown in Figure 5.

Crucially, we run 64 trials using different initial conditions, random vectors drawn from a normal distribution in the VAE latent space, to arrive at 64 different optimized devices for each problem, from which we then choose the best performing one as the “champion device” (see Figure 5a,e). We see that many runs converge to values significantly worse than the best one, i.e. the optimization is highly sensitive to the choice of initial parameters, which is a problem that any local optimization

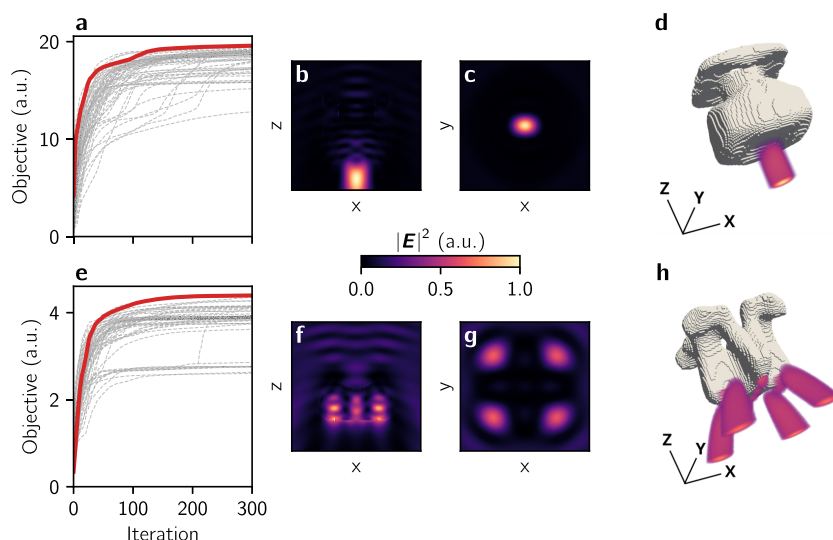


Figure 5. Inverse-design using FNO-3D of (a)–(d), a nanophotonic device that focuses into a single spot, and (e)–(h), a nanophotonic device that focuses into four focal spots under plane-wave illumination. We show the (a, e) respective optimization histories of 64 optimizations for each device with different random initial conditions. The red line marks the best final device. Slices of the electric field intensity $|E|^2$ for the best devices, as obtained from full-wave FDTD simulations, are shown in the (b, f) x – z plane as well as in the (c, g) focal x – y plane. Images (d) and (h) show three-dimensional views of both devices along with a volume rendering of the field intensity thresholded such that only the highest-intensity regions are visible.

faces. However, sampling many different initial configurations is often not feasible for an inverse design using full-wave solvers, and only a few, if any, additional trials are typically performed. In contrast, a single FNO-3D optimization takes around 10 min (≈ 2 seconds per iteration, NVIDIA A100 SXM4 GPU), and two optimizations can be run in parallel on a single GPU. We run the optimizations for all 64 trials in parallel on 32 GPUs so that each full optimization run still takes only 10 min. This, of course, depends on the computational facilities at hand, but even running all optimizations serially would take just over 5 h. For comparison, a single such optimization would take several days (20 min per simulation, two simulations per iteration, see the data generation section), rendering a sweep over many different initial conditions such as the one demonstrated here practically infeasible.

We perform full-wave simulations to evaluate the performance of the final optimized devices. For the lens shown in Figure 5d, the L_1 error between simulated and predicted field intensity $|E|^2$ is 10.2%, and for the 4-point lens in Figure 5h, the same error is 9.1%. While this is slightly higher than the error on the test set, it is still well within reasonable accuracy, in particular regarding the uncertainty of microfabrication technologies, e.g., 3D laser nanoprinting, that would be used to realize devices like this. The reason for the increased error when compared to the test error is simple, the error on the squared absolute fields is necessarily larger. In fact, the L_1 losses on $|E|$ are in line with those of the test set means, 5.8% for the single lens and 5.0% for the 4-point lens. Additionally, we note that the error in the intensity comes mainly from the absolute numerical values of the field components; the qualitative agreement between FDTD and FNO-3D fields remains high, and we can see that both devices fulfill their respective design goals. All fields shown in Figure 5 are taken from full-wave FDTD simulations.

DISCUSSION

An (on the surface) compelling advantage of using surrogate models for solving scattering problems is the almost negligible inference time required when compared to running a full-wave solver. However, there are some major concessions that lead to this speedup, and it requires careful consideration whether this trade-off is acceptable for a given task or not. The most obvious caveats are perhaps generalization and accuracy; outside of very specific cases,⁵⁶ a surrogate solver trained on finite data will not generalize to data that lies outside of the distribution that it was trained on, and the inference error will only approach zero given sufficient network expressivity,⁵⁷ as well as training set size.⁵⁸ More concretely, this means that for optical simulations, a data-driven model can only “solve” certain classes of scattering problems, namely, those that it was trained on, e.g., specific source distributions, materials, and domain sizes, to name a few. These limitations can generally be overcome to a degree by choosing a suitable network architecture, increasing the variety of training samples, and increasing the total amount of training data, leading to better generalization and increased accuracy. However, if the cost of generating sufficient training data exceeds the cost of solving the task using conventional methods, then there seems to be little practical benefit of training a surrogate model. As long as data-driven methods rely on data that is generated by classical methods, the latter can neither be “superseded” nor are they, in many cases, slower when considered in the proper context.

We illustrate this in Figure 6, where we show the total time taken for both FDTD and FNO-3D simulations and the time needed to generate FNO-3D training data. Note that timings can vary considerably between different hardware and simulation setups, and the values shown in Figure 6 should only be seen within the context of this work. FNO-3D is indeed 3 orders of magnitude faster than FDTD during inference; however, we argue that while this might be impressive when looked at in isolation, it is not a particularly relevant metric when making comparisons to conventional

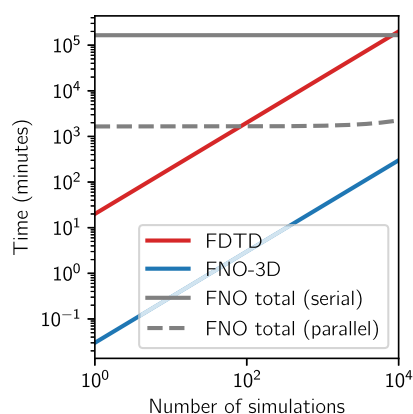


Figure 6. Comparison of the time required for full-wave simulations (FDTD) and FNO-3D predictions for a varying number of simulations. Time for a single simulation is assumed to be 20 min. Also shown is the total time of FNO-3D inference in addition to sample generation (8192 samples) and network training (≈ 24 hours) in the case of serial data generation (solid gray line) and parallel data generation (dashed gray line) used in this work.

solvers. The bulk of the time spent lies in data generation, in addition to training and inference time, both of which are practically negligible in comparison. An advantage of data-driven approaches lies in the fact that individual samples are independent, i.e., data generation can happen in parallel and is only practically limited by the computational resources at hand. As an upper bound, and including training time, a surrogate model needs to be used for inference as many times as the number of samples contained in the data set that it was trained on to just break even with a full-wave solver. Beyond this point, a surrogate solver becomes cheaper concerning both total time and computational resources, i.e., energy. With respect to total time, this bound can be lowered considerably by generating samples in parallel as we have done here, see the data generation section.

All this is to say that data efficiency is crucial in the context of scientific machine learning in general and for surrogate solvers in particular, and it needs to be addressed critically and transparently.⁵⁹ We identify three aspects that should be considered when using deep learning in this context: architecture, specialization, and application. By choosing a suitable model architecture, the amount of training data required to reach sufficiently small test errors can be reduced significantly, as we demonstrate in the field inference section for an electromagnetic scattering problem, and similar observations have been made across a variety of physical domains.^{38,60} Physics-informed approaches that incorporate governing equations into the model's loss function^{29,33,61,62} similarly contribute to higher data efficiency. Second, it is practical to aim for model specialization, i.e., a surrogate solver does not need to generalize to all possible problem configurations, as such generalization tends to lead to unjustifiable data requirements. To keep data requirements minimal, it is expedient to limit the scope of the model's intended application. Lastly, the application needs to benefit from the use of a surrogate solver in a way that justifies the cost of training. We demonstrate this in the inverse design section by using a surrogate solver to perform a free-form, three-dimensional inverse design of electromagnetic scatterers. Here, we train the model on a data set of 8192 samples and run a total of 128 independent optimizations with 300 iterations

each. A comparable gradient-based approach using the adjoint method would equate to running $128 \times 300 \times 2 = 76800$ full-wave simulations, far exceeding the number of simulations performed for generating the training data. Lastly, we would like to stress that while surrogate models can be useful in many cases and will undoubtedly play an important role in the future, fast "classical", e.g., semianalytical, methods should generally be preferred if the problem at hand allows for their application as they often offer comparable speed at much higher accuracy and have well-controlled error bounds.

In summary, we have demonstrated the use of a neural operator-based model for solving electromagnetic scattering problems and show that it outperforms current state-of-the-art by a significant margin. As a data-driven method, the surrogate solver inherently suffers from a loss of accuracy and generality when compared to full-wave solvers. Nonetheless, we demonstrate that this approach can be used for complex tasks such as the free-form, gradient-based inverse design of three-dimensional electromagnetic scatterers. Machine learning-based surrogate solvers have the potential to be highly useful for applications in nanophotonics. However, it is essential to find efficient model architectures and identify tasks well-suited for their application. We believe that our work contributes to both of these aspects and look forward to future developments in this field.

METHODS

Data Generation. To generate a large number of random scatterer geometries, we employ the same method as presented in Repán et al.⁶³ We sample points from a random uniform distribution on the interval $[0, 1)$ on a regular square (cubic in 3D) grid with a side length of 128 px. A zero-padded Gaussian blur ($\sigma = 12$ px) is then applied to the whole grid, and the result is thresholded at a value of 0.5. The zero-padding ensures that the scatterers are fully contained within the simulation domain and do not extend into the boundaries. This procedure leads to a diverse set of smooth, random geometries containing one or multiple scatterers. The whole data generation process is illustrated in Figure 7. We stress that the scatterers generated by this method can be almost

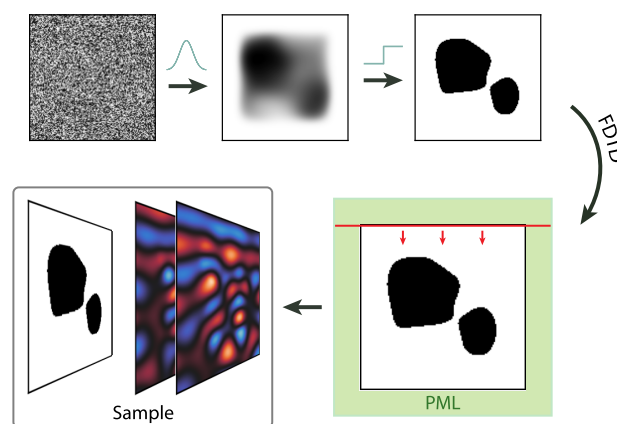
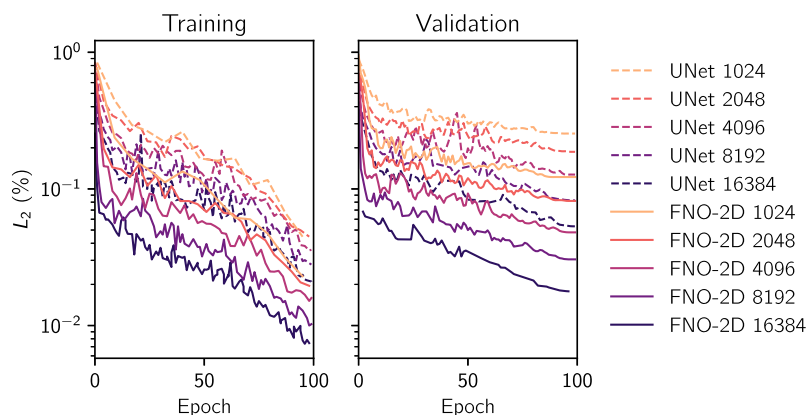


Figure 7. Illustration of the data generation process. A binary image is generated by smoothing and thresholding an image drawn from a random uniform distribution. The image is then interpreted as a material distribution and simulated with plane-wave illumination using FDTD. The sample for the training set is then assembled from the scatterer (input) and the individual field components (target).

Table 3. Summary of the Datasets of Random Scattering Geometries Used for Training All Models

type	samples	input data	output data	input shape	output shape
2D	17040	scatterer pixels	E_z	128×128	$2 \times 128 \times 128$
3D	8720	scatterer voxels	E_x, E_y, E_z	$128 \times 128 \times 128$	$6 \times 128 \times 128 \times 128$

**Figure 8.** Training and validation L_2 loss curves for all UNet and FNO-2D training runs on different data set sizes.

arbitrarily complex, and the size of the Gaussian smoothing kernel primarily determines this complexity. The size of the smoothing is directly analogous to the “filtering” method when choosing a minimal feature size in topology optimization.^{44,64}

The generated random scatterers are interpreted as a material distribution, with 1 indicating the presence of material ($n_{\text{high}} = 1.5$) and 0 indicating air ($n_{\text{low}} = 1$) and illuminated using a plane-wave source at a wavelength of $\lambda_0 = 1 \mu\text{m}$. We choose $n_{\text{high}} = 1.5$, as it corresponds roughly to the typical polymers used in 3D laser nanoprinting. We run the simulations at a spatial resolution of $25 \text{ px}/\mu\text{m}$, which equates to a spatial extent of $5.12 \mu\text{m}$ along each axis. Additionally, the simulation domain is surrounded by perfectly matched layers (PMLs) with a thickness of $0.5 \mu\text{m}$ on each side, increasing the side length of the simulation domain to $6.12 \mu\text{m}$. The data set comprises pairs of steady-state electric fields (excluding PML regions) and the corresponding scatterer. The simulations for both the 2D and 3D data sets are performed using the open-source finite-difference time-domain (FDTD) software package Meep.⁵ A summary of the data sets is given in Table 3.

The total size of the 2D data set is made up of 16384 (2^{14}) training samples, 256 validation samples, and 400 test samples. Smaller 2D data sets used in the field inference section were subsampled from this larger data set.

The 3D data set is made up of 8192 training samples, 128 validation samples, and 400 test samples. Each sample took roughly 20 min (with some slight variations due to different scatterers) to simulate on four cores of an Intel Xeon Platinum 8368 CPU (76 cores total). The data was generated on 40 nodes of the HoreKa cluster, with each node simulating 19 samples in parallel at a time. In total, the generation of the 3D data set took just under 4 h.

FNO and UNet Training. Details on the FNO hyperparameters are given in Table 1. The UNet model consists of five downsampling (max pooling) and five upsampling blocks, where each block comprises six convolutional layers with batch normalization and ReLU activation. We do not implement UNet ourselves in this work, instead, please refer to Chen et al.²⁹ for a detailed overview of the architecture. We deviate slightly from their model by replacing the periodic padding in the convolutional layers with zero padding, as we have found

this to increase UNet performance on our data set by $\approx 1\%$ across all runs.

All models are trained for 100 epochs using AdamW⁵³ with a one-cycle⁶⁵ learning rate policy and a batch size of 32. We use the relative L_2 error (eq 4, $p = 2$) as the training loss function and monitor the relative L_1 and L_2 errors during validation at the end of each epoch with a validation split size of 256 samples. Training and validation loss curves are shown in Figure 8. The FNO-2D and UNet models are trained on a single NVIDIA A100 SXM4 GPU. The training time per epoch is summarized in Table 2 for each data set size.

For training the FNO-3D model, we use the same effective batch size as for the 2D models. This does not, however, fit into GPU memory during training, so we train the model in parallel across two nodes with 4 GPUs each, where each GPU only operates on 4 data samples. To further reduce memory requirements, we also use activation checkpointing⁶⁶ on the FNO blocks in the model. After each iteration, the gradients are synchronized and averaged across all processes using PyTorch’s DistributedDataParallel computing model.⁶⁷ The total training time for the FNO-3D model was 11 h. The training hyperparameters are summarized in Table 4.

Table 4. Training Hyperparameters Used for the Surrogate Solver Models

model	parameters	learning rate (min/max)	batch size	validation split
UNet	23617970	5e-5/5e-4	32	256
FNO-2D	5909250	$10^{-3}/10^{-2}$	32	256
FNO-3D	141568902	$10^{-3}/10^{-2}$	$2 \times 4 \times 4^a$	128

^aTrained on two nodes with 4 GPUs each and four samples per GPU (NVIDIA A100 SXM4).

Note that UNets are trained using a smaller learning rate than FNO, this is because UNet training becomes unstable at a significantly lower learning rate than FNO. To perform a fair comparison, we ran hyperparameter sweeps over multiple learning rates as well as learning rate schedulers for UNet. The models presented herein always represent the best-performing networks that we found.

VAE Setup and Training. The VAE used in this work is based on a convolutional encoder–decoder-type architecture. The encoder contains five downsampling blocks that compress the input (a $128 \times 128 \times 128$ image of the material distribution) into a latent space representation. Each downsampling block consists of a 3D convolution, a batch normalization layer, and SELU⁶⁸ activation. For the convolutional layers, we use a kernel size of 5, a stride of 2, and a padding of 2, while doubling the number of channels in each layer. The decoder mirrors the layers of the encoder, where the convolutional layers are replaced with transposed convolutions⁶⁹ for upsampling. The VAE network contains 86433514 trainable parameters in total.

The size of the latent space is, in principle, arbitrary and depends on the complexity of the geometries that should be modeled; the more complex the geometries, the larger the latent space needs to be to represent all possible topologies. We have found a latent space vector with 2048 elements to lead to good results during optimization for our random scattering geometries. While VAEs with smaller latent sizes (down to around 256 elements) still show good reconstruction capabilities during training, we have found these to give worse results during inverse design, with less varied shapes and generally lower fidelity. We suspect that while one might be able to reconstruct an input with a relatively small latent space adequately, the separation of different geometries within that space might be rather large and, thus, hard to reach for a gradient-based optimizer. Suppose one chooses a larger latent space than what is strictly needed for reconstruction. In that case, the distance between different classes of geometries shrinks in this higher dimensional space, and an optimization might reach them more easily.

For training, we minimize the evidence lower bound (ELBO)⁵² with an additional penalty term for binarization:

$$L_{\text{VAE}}(x, \hat{x}) = \underbrace{\gamma_{\text{KL}} L_{\text{KL}}(x, \hat{x}) - L_{\text{R}}(x, \hat{x})}_{\text{ELBO}} - \gamma_{\text{B}} L_{\text{B}} \quad (6)$$

with the original image x , the reconstructed image $\hat{x}$, the Kullback–Leibler divergence L_{KL} , the reconstruction loss L_{R} , and the binarization penalty L_{B} :

$$L_{\text{KL}} = \mathbb{E}_q[\log q(z|x) - \log p(z)] \quad (7)$$

$$L_{\text{R}} = \mathbb{E}_q[\log p(x|z)] \quad (8)$$

$$L_{\text{B}} = \frac{1}{3} \min \left[-\log_{10} \left(\frac{1}{N} \sum_i^N 4x_i(1 - x_i) \right), 3 \right] \quad (9)$$

where p is a standard normal distribution, q is the distribution parametrized by the encoder output given a sample x , and z is a sample drawn from q . To balance the three terms of the VAE loss, we introduce the annealing parameters γ_{KL} and γ_{B} that increase the weighting of L_{KL} and L_{B} at different points in the training, respectively. The training loss curves are presented in Figure 9.

We do not pregenerate a data set for VAE training. Instead, we use the geometry sampling procedure outlined in the data generation section to generate images of random scatterers on-the-fly during training. The network is trained using a batch size of 64, for a total of 60000 batches, resulting in a total training time of 48 h (single NVIDIA A100 SXM4 GPU).

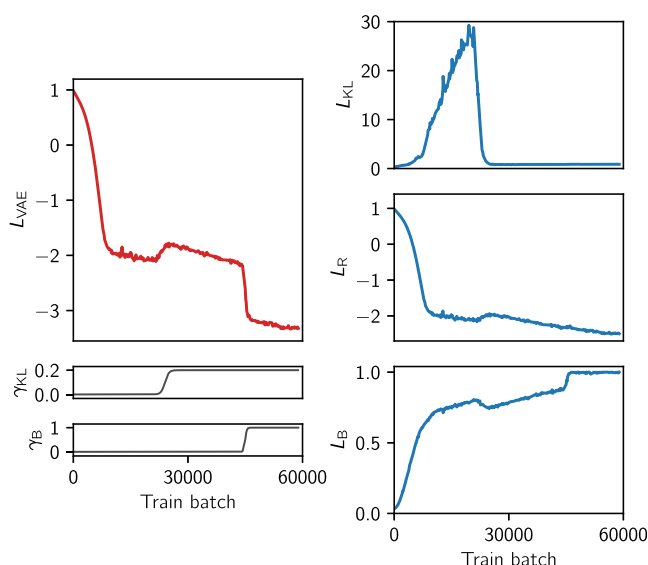


Figure 9. Total loss L_{VAE} during training together with the annealed weight parameters γ_{KL} and γ_{B} (left) as well as the individual components of L_{VAE} for KL-divergence L_{KL} , reconstruction L_{R} , and binarization L_{B} (right).

■ ASSOCIATED CONTENT

Data Availability Statement

All code is made freely available under https://github.com/tfp-photonics/neurop_invdes. This includes the code for data generation, network training, and inverse design, as well as the model implementations. We publish our research data under DOI: 10.35097/911, which includes the generated data sets and model weights as well as the code and data used for generating Figures 2, 3, 5, 6, 8, and 9.

■ AUTHOR INFORMATION

Corresponding Authors

Yannick Augenstein – Institute of Theoretical Solid State Physics, Karlsruhe Institute of Technology, 76131 Karlsruhe, Germany; orcid.org/0000-0002-0045-9407; Email: yannick.augenstein@kit.edu

Carsten Rockstuhl – Institute of Theoretical Solid State Physics, Karlsruhe Institute of Technology, 76131 Karlsruhe, Germany; Institute of Nanotechnology, Karlsruhe Institute of Technology, 76021 Karlsruhe, Germany; Email: carsten.rockstuhl@kit.edu

Author

Taavi Repän – Institute of Physics, University of Tartu, Tartu 50411, Estonia; orcid.org/0000-0001-6596-2022

Complete contact information is available at: <https://pubs.acs.org/10.1021/acsp Photonics.3c00156>

Funding

This research has been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy via the Excellence Cluster 3D Matter Made to Order (EXC-2082/1, Grant No. 390761711). Y.A. acknowledges support from the Carl Zeiss Foundation via the CZFFocus@HEiKA program. T.R. is supported by the Estonian Research Council (Grant No. PSG716). The authors acknowledge support by the state of Baden-Württemberg through bwHPC and the German Research Foundation

(DFG) through Grant No. INST 40/575-1 FUGG (JUSTUS 2 cluster). The computations involved in this work were partially performed on the HoreKa supercomputer funded by the Ministry of Science, Research and the Arts Baden-Württemberg and by the Federal Ministry of Education and Research.

Notes

The authors declare no competing financial interest.

ACKNOWLEDGMENTS

The authors kindly thank Lina Kuhn for helpful discussions regarding variational autoencoders.

REFERENCES

- (1) Volakis, J. L.; Volakis, J. L.; Chatterjee, A.; Kempel, L. C. *Finite Element Method Electromagnetics: Antennas, Microwave Circuits, and Scattering Applications*; Wiley-IEEE Press, 1998.
- (2) Jin, J.-M. *The Finite Element Method in Electromagnetics*; John Wiley & Sons, 2015.
- (3) Zhao, Y.-J.; Wu, K.-L.; Cheng, K.-K. M. A compact 2-D full-wave finite-difference frequency-domain method for general guided wave structures. *IEEE Transactions on Microwave Theory and Techniques* **2002**, *50*, 1844–1848.
- (4) Hughes, T. W.; Williamson, I. A.; Minkov, M.; Fan, S. Forward-Mode Differentiation of Maxwell's Equations. *ACS Photonics* **2019**, *6*, 3010–3016.
- (5) Oskooi, A. F.; Roundy, D.; Ibanescu, M.; Bermel, P.; Joannopoulos, J. D.; Johnson, S. G. MEEP: A flexible free-software package for electromagnetic simulations by the FDTD method. *Comput. Phys. Commun.* **2010**, *181*, 687–702.
- (6) Vaccari, A.; Cala'Lesina, A.; Cristoforetti, L.; Pontalti, R. Parallel Implementation of a 3D Subgridding FDTD Algorithm for Large Simulations. *Progress in Electromagnetics Research* **2011**, *120*, 263–292.
- (7) Taflov, A.; Oskooi, A.; Johnson, S. G. *Advances in FDTD Computational Electrodynamics: Photonics and Nanotechnology*; Artech House, 2013.
- (8) Molesky, S.; Lin, Z.; Piggott, A. Y.; Jin, W.; Vucković, J.; Rodriguez, A. W. Inverse design in nanophotonics. *Nat. Photonics* **2018**, *12*, 659–670.
- (9) Hughes, T. W.; Minkov, M.; Williamson, I. A.; Fan, S. Adjoint Method and Inverse Design for Nonlinear Nanophotonic Devices. *ACS Photonics* **2018**, *5*, 4781–4787.
- (10) Schneider, P.-I.; Garcia Santiago, X.; Soltwisch, V.; Hammerschmidt, M.; Burger, S.; Rockstuhl, C. Benchmarking Five Global Optimization Approaches for Nano-optical Shape Optimization and Parameter Reconstruction. *ACS Photonics* **2019**, *6*, 2726–2733.
- (11) Augenstein, Y.; Rockstuhl, C. Inverse Design of Nanophotonic Devices with Structural Integrity. *ACS Photonics* **2020**, *7*, 2190–2196.
- (12) Lin, Z.; Roques-Carnes, C.; Pestourie, R.; Soljačić, M.; Majumdar, A.; Johnson, S. G. End-to-end nanophotonic inverse design for imaging and polarimetry. *Nanophotonics* **2021**, *10*, 1177–1187.
- (13) Hughes, T. W.; Minkov, M.; Liu, V.; Yu, Z.; Fan, S. A perspective on the pathway toward full wave simulation of large area metalenses. *Appl. Phys. Lett.* **2021**, *119*, 150502.
- (14) Lin, H.-C.; Wang, Z.; Hsu, C. W. Fast multi-source nanophotonic simulations using augmented partial factorization. *Nature Computational Science* **2022**, *2*, 815–822.
- (15) Li, L. New formulation of the Fourier modal method for crossed surface-relief gratings. *Journal of the Optical Society of America A* **1997**, *14*, 2758–2767.
- (16) Liu, V.; Fan, S. S4: A free electromagnetic solver for layered periodic structures. *Comput. Phys. Commun.* **2012**, *183*, 2233–2244.
- (17) Minkov, M.; Williamson, I. A.; Andreani, L. C.; Gerace, D.; Lou, B.; Song, A. Y.; Hughes, T. W.; Fan, S. Inverse Design of Photonic Crystals through Automatic Differentiation. *ACS Photonics* **2020**, *7*, 1729–1741.
- (18) Beutel, D.; Groner, A.; Rockstuhl, C.; Fernandez-Corbaton, I. Efficient simulation of bi-periodic, layered structures based on the T-matrix method. *Journal of the Optical Society of America B* **2021**, *38*, 1782–1791.
- (19) Pestourie, R.; Mroueh, Y.; Rackauckas, C.; Das, P.; Johnson, S. G. Physics-enhanced deep surrogates for PDEs. *arXiv:2111.05841 [cs.LG]* **2021**, na.
- (20) Lu, L.; Jin, P.; Pang, G.; Zhang, Z.; Karniadakis, G. E. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence* **2021**, *3*, 218–229.
- (21) Chen, X.; Chen, R.; Wan, Q.; Xu, R.; Liu, J. An improved data-free surrogate model for solving partial differential equations using deep neural networks. *Sci. Rep.* **2021**, *11*, 19507.
- (22) Hornik, K.; Stinchcombe, M.; White, H. Multilayer feedforward networks are universal approximators. *Neural Networks* **1989**, *2*, 359–366.
- (23) Baxter, J.; Desautels, J.; Lesina, A. C.; Berini, P.; Ramunno, L. Deep Learning for Engineering Optical Scattering from Plasmonic Nanostructures. *OSA Technical Digest; Optica Publishing Group*, 2021; p JW2D.4. DOI: 10.1364/FLATOPTICS.2021.JW2D.4.
- (24) Krasikov, S.; Tranter, A.; Bogdanov, A.; Kivshar, Y. Intelligent metaphotonics empowered by machine learning. *Opto-Electronic Advances* **2022**, *5*, 210147–1.
- (25) An, S.; Fowler, C.; Zheng, B.; Shalaginov, M. Y.; Tang, H.; Li, H.; Zhou, L.; Ding, J.; Agarwal, A. M.; Rivero-Baleine, C.; et al. A Deep Learning Approach for Objective-Driven All-Dielectric Metasurface Design. *ACS Photonics* **2019**, *6*, 3196–3207.
- (26) Jiang, J.; Fan, J. A. Simulator-based training of generative neural networks for the inverse design of metasurfaces. *Nanophotonics* **2019**, *9*, 1059–1069.
- (27) Wiecha, P. R.; Muskens, O. L. Deep Learning Meets Nanophotonics: A Generalized Accurate Predictor for Near Fields and Far Fields of Arbitrary 3D Nanostructures. *Nano Lett.* **2020**, *20*, 329–338.
- (28) Pestourie, R.; Mroueh, Y.; Nguyen, T. V.; Das, P.; Johnson, S. G. Active learning of deep surrogates for PDEs: application to metasurface design. *npj Computational Materials* **2020**, *6*, 164.
- (29) Chen, M.; Lupo, R.; Mao, C.; Huang, D.-H.; Jiang, J.; Lalanne, P.; Fan, J. A. High Speed Simulation and Freeform Optimization of Nanophotonic Devices with Physics-Augmented Deep Learning. *ACS Photonics* **2022**, *9*, 3110–3123.
- (30) Karniadakis, G. E.; Kevrekidis, I. G.; Lu, L.; Perdikaris, P.; Wang, S.; Yang, L. Physics-informed machine learning. *Nature Reviews Physics* **2021**, *3*, 422–440.
- (31) Lu, L.; Pestourie, R.; Yao, W.; Wang, Z.; Verdugo, F.; Johnson, S. G. Physics-Informed Neural Networks with Hard Constraints for Inverse Design. *SIAM Journal on Scientific Computing* **2021**, *43*, B1105–B1132.
- (32) Cuomo, S.; Di Cola, V. S.; Giampaolo, F.; Rozza, G.; Raissi, M.; Piccialli, F. Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What's Next. *Journal of Scientific Computing* **2022**, *92*, 88.
- (33) Lim, J.; Psaltis, D. MaxwellNet: Physics-driven deep neural network training based on Maxwell's equations. *APL Photonics* **2022**, *7*, 011301.
- (34) Li, Z.; Kovachki, N.; Azizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Neural Operator: Graph Kernel Network for Partial Differential Equations. *arXiv:2003.03485 [cs.LG]* **2020**, na.
- (35) Li, Z.; Kovachki, N.; Azizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Fourier Neural Operator for Parametric Partial Differential Equations. *arXiv:2010.08895 [cs.LG]* **2021**, na.
- (36) Kovachki, N.; Li, Z.; Liu, B.; Azizzadenesheli, K.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Neural Operator: Learning Maps Between Function Spaces. *arXiv:2108.08481 [cs.LG]* **2022**, na.

- (37) Lu, L.; Meng, X.; Cai, S.; Mao, Z.; Goswami, S.; Zhang, Z.; Karniadakis, G. E. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering* **2022**, *393*, 114778.
- (38) Lu, L.; Pestourie, R.; Johnson, S. G.; Romano, G. Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport. *Physical Review Research* **2022**, *4*, 023210.
- (39) Elsayy, M. M.; Gourdin, A.; Binois, M.; Duvigneau, R.; Felbacq, D.; Khadir, S.; Genevet, P.; Lanteri, S. Multiobjective Statistical Learning Optimization of RGB Metalens. *ACS Photonics* **2021**, *8*, 2498–2508.
- (40) Roques-Carmes, C.; Lin, Z.; Christiansen, R. E.; Salamin, Y.; Kooi, S. E.; Joannopoulos, J. D.; Johnson, S. G.; Soljacic, M. Toward 3D-Printed Inverse-Designed Metaoptics. *ACS Photonics* **2022**, *9*, 43–51.
- (41) Wiecha, P. R.; Petrov, A. Y.; Genevet, P.; Bogdanov, A. Inverse design of nanophotonics devices and materials. *Photonics and Nanostructures – Fundamentals and Applications* **2022**, *52*, 101084.
- (42) Yao, W.; Verdugo, F.; Christiansen, R. E.; Johnson, S. G. Trace formulation for photonic inverse design with incoherent sources. *Structural and Multidisciplinary Optimization* **2022**, *65*, 336.
- (43) Chen, M.; Jiang, J.; Fan, J. A. Algorithm-Driven Paradigms for Freeform Optical Engineering. *ACS Photonics* **2022**, *9*, 2860–2871.
- (44) Sigmund, O.; Maute, K. Topology optimization approaches. *Structural and Multidisciplinary Optimization* **2013**, *48*, 1031–1055.
- (45) Hammond, A. M.; Oskooi, A.; Chen, M.; Lin, Z.; Johnson, S. G.; Ralph, S. E. High-performance hybrid time/frequency-domain topology optimization for large-scale photonics inverse design. *Opt. Express* **2022**, *30*, 4467–4491.
- (46) Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; Appleton, G.; Axton, M.; Baak, A.; Blomberg, N.; Boiten, J.-W.; da Silva Santos, L. B.; Bourne, P. E.; et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* **2016**, *3*, 1–9.
- (47) Ioffe, S.; Szegedy, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning* **2015**, 448–456.
- (48) Hendrycks, D.; Gimpel, K. Gaussian Error Linear Units (GELUs). *arXiv:1606.08415 [cs.LG]* **2020**, na.
- (49) Ronneberger, O.; Fischer, P.; Brox, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. *International Conference on Medical Image Computing and Computer-Assisted Intervention; MICCAI*, 2015; pp 234–241.
- (50) Allen, M.; Poggiali, D.; Whitaker, K.; Marshall, T. R.; Kievit, R. A. Raincloud plots: a multi-platform tool for robust data visualization. *Wellcome Open Research* **2019**, *4*, 63.
- (51) Scott, D. W. On optimal and data-based histograms. *Biometrika* **1979**, *66*, 605–610.
- (52) Hendrycks, D.; Gimpel, K. Auto-Encoding Variational Bayes. *arXiv:1312.6114 [stat.ML]* **2022**, na.
- (53) Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. *arXiv:1711.05101 [cs.LG]* **2019**, na.
- (54) Zhu, C.; Byrd, R. H.; Lu, P.; Nocedal, J. Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. *ACM Transactions on Mathematical Software* **1997**, *23*, 550–560.
- (55) Svanberg, K. The method of moving asymptotes – a new method for structural optimization. *International Journal for Numerical Methods in Engineering* **1987**, *24*, 359–373.
- (56) Power, A.; Burda, Y.; Edwards, H.; Babuschkin, I.; Misra, V. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. *arXiv:2201.02177 [cs.LG]* **2022**, na.
- (57) Raghu, M.; Poole, B.; Kleinberg, J.; Ganguli, S.; Sohl-Dickstein, J. On the Expressive Power of Deep Neural Networks. *International Conference on Machine Learning; ICML*, 2017; pp 2847–2854.
- (58) Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press, 2016.
- (59) Woldseth, R. V.; Aage, N.; Bærentzen, J. A.; Sigmund, O. On the use of artificial neural networks in topology optimization. *Structural and Multidisciplinary Optimization* **2022**, *65*, 294.
- (60) Cai, S.; Wang, Z.; Lu, L.; Zaki, T. A.; Karniadakis, G. E. DeepM&Mnet: Inferring the electroconvection multiphysics fields based on operator approximation by neural networks. *J. Comput. Phys.* **2021**, *436*, 110296.
- (61) Wang, S.; Wang, H.; Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances* **2021**, *7*, No. eabi8605.
- (62) Goswami, S.; Yin, M.; Yu, Y.; Karniadakis, G. E. A physics-informed variational DeepONet for predicting crack path in quasi-brittle materials. *Computer Methods in Applied Mechanics and Engineering* **2022**, *391*, 114587.
- (63) Repán, T.; Augenstein, Y.; Rockstuhl, C. Exploiting geometric biases in inverse nano-optical problems using artificial neural networks. *Opt. Express* **2022**, *30*, 45365–45375.
- (64) Wang, F.; Lazarov, B. S.; Sigmund, O. On projection methods, convergence and robust formulations in topology optimization. *Structural and Multidisciplinary Optimization* **2011**, *43*, 767–784.
- (65) Smith, L. N.; Topin, N. Super-convergence: very fast training of neural networks using large learning rates. *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications; SPIE*, 2019; pp 369–386.
- (66) FairScale authors FairScale: A general purpose modular PyTorch library for high performance and large scale training. <https://github.com/facebookresearch/fairscale>, 2021; accessed: 2023-03-22.
- (67) Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* **2019**, *32*, 8026–8037.
- (68) Klambauer, G.; Unterthiner, T.; Mayr, A.; Hochreiter, S. Self-Normalizing Neural Networks. *arXiv:1706.02515 [cs.LG]* **2017**, na.
- (69) Zeiler, M. D.; Krishnan, D.; Taylor, G. W.; Fergus, R. Deconvolutional Networks. *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition; IEEE*, 2010; pp 2528–2535.



CAS BIOFINDER DISCOVERY PLATFORM™

CAS BIOFINDER HELPS YOU FIND YOUR NEXT BREAKTHROUGH FASTER

Navigate pathways, targets, and
diseases with precision

Explore CAS BioFinder

